

国家级精品课程
山东省一流本科课程



程序设计基础（C语言）

信息科学与工程学院

C语言课程组

第八章 文件

- ❖ 8.1 文件概述
- ❖ 8.2 文件的打开与关闭
- ❖ 8.3 文件的读写
- ❖ 8.4 文件的定位
- ❖ 8.5 文件检测函数

8.1 文件概述

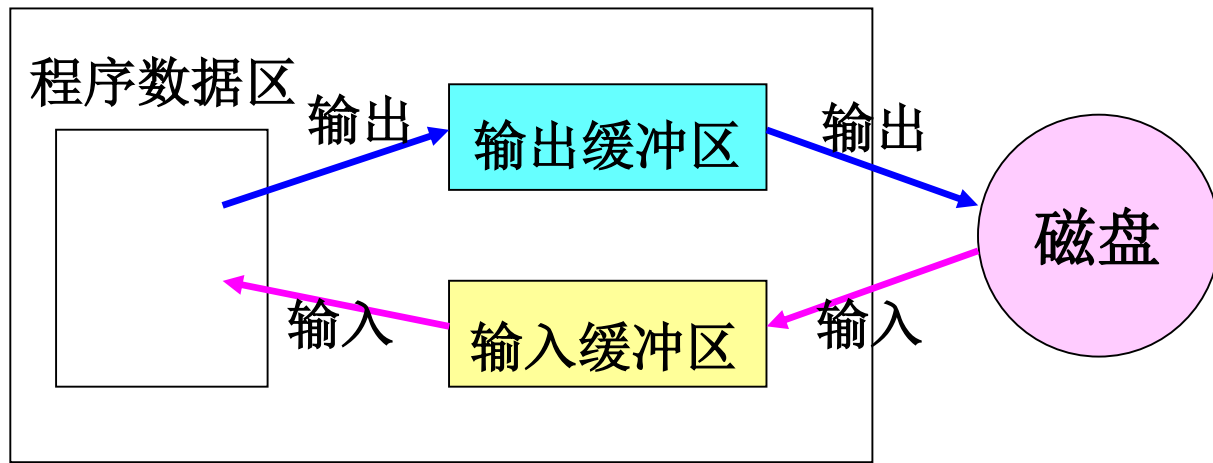
- 在我们目前所编的程序中，数据通过键盘输入，存储在变量、数组或链表中，但这些数据在程序结束后就消失了。为了在程序结束后将有用的数据保存下来，就需要使用文件。
 - 计算机可以把文件长期存放在外部存储设备中，如硬盘、磁盘、光盘、U盘、磁带等，在我们需要使用数据时，从这些存储设备上读取数据
 - ◆ 文件: 指存储在外部介质上的有序数据的集合
- 操作系统是**以文件为单位**对数据进行管理的，现代操作系统把所有外部设备都认为是文件，以便进行统一的管理。
- C语言认为文件是磁盘文件和其它具有输入输出(I/O)功能的外部设备(如键盘、显示器等)的总称。

8.1 文件概述

◆ 磁盘文件系统对文件的处理方法

- 向磁盘文件输出数据时(即写文件), 先将程序中变量的值送到“输出缓冲区”, 等缓冲区装满后再将数据一起传送给磁盘文件
- 从磁盘文件输入数据时(即读文件), 先将数据送到“输入缓冲区”,

再从缓冲区将数据传送给程序数据区(即传给程序中的变量)



使用缓冲区的目的是为了减少对磁盘的实际读写次数

8.1.1 文件的分类

- ◆ C语言把文件看作是一个字节的序列, 根据数据的组织形式把文件分为两类: **文本文件和二进制文件**。

1. 文本文件(也称ASCII文件)

文本文件的每一个字节存放一个字符, 每个字符用一个ASCII码表示。

例: 整数2460在文本文件中占用4个字节, 分别存放字符'2', '4', '6', '0'

字符' 2' 字符' 4' 字符' 6' 字符' 0'

0011 0010	0011 0100	0011 0110	0011 0000
-----------	-----------	-----------	-----------

字符' 2' 的ASCII码值为50, $2^5+2^4+2^1=32+16+2=50$

字符' 4' 的ASCII码值为52, $2^5+2^4+2^2=32+16+4=52$

字符' 6' 的ASCII码值为54, $2^5+2^4+2^2+2^1=32+16+4+2=54$

字符' 0' 的ASCII码值为48, $2^5+2^4=32+16=48$

8.1.1 文件的分类

2. 二进制文件

二进制文件则是以字节为单位存放数据的二进制代码，将存储的信息严格按其在内存中的存储形式来保存。

例：整数2460，它在内存中的存储形式：

0000 1001	1001 1100
-----------	-----------

$$2460 = 2^{11} + 2^8 + 2^7 + 2^4 + 2^3 + 2^2 = 2048 + 256 + 128 + 16 + 8 + 4$$

2460在二进制文件中的存储形式：

0000 1001	1001 1100
-----------	-----------



8.1.1 文件的分类

◆ 文本文件和二进制文件的优缺点

● 文本文件

优点: 便于对字符进行处理;便于在文本编辑器中直接阅读

缺点: 占用存储空间较多, 计算机处理数据时需要将ASCII码形式转换成二进制形式, 会花费较多的时间, 降低了程序的执行效率。

● 二进制文件

优点: 节省存储空间, 无需转换时间, 程序执行效率较高

缺点: 不能直接输出字符形式, 可读性差。



8.1.2 文件类型指针

C语言中定义了一个结构体数据类型FILE来描述文件信息

```
struct _iobuf    //VC++6.0的“stdio.h”中FILE的定义
{
    char *_ptr;    //文件输入的下一个位置
    int _cnt;      //当前缓冲区的相对位置
    char *_base;   //文件的起始位置
    int _flag;     //文件标志
    int _file;     //文件的有效性验证
    int _charbuf;  //检查缓冲区状况,若无缓冲区则不读取
    int _bufsiz;   //缓冲区大小
    char *_tmpfname; //临时文件名
};
typedef struct _iobuf FILE;
```

- FILE类型中的每个成员是用来存放有关文件的各种信息的数据项。
(注意：一般情况下程序员不会直接使用FILE中的成员)



8.1.2 文件类型指针

- 在程序中每一个打开的文件都必须有一个文件指针变量，该指针变量用来存储文件的基本信息，实现对文件的操作。
- ◆ 文件指针变量的定义形式：**FILE * 文件指针变量名**；

注意：

1. 只有通过文件指针变量才能调用相应的文件
2. 在程序中若同时使用n个文件就要定义n个文件指针变量，分别对应各个文件
3. **FILE**必须大写

8.1.3 文件操作的基本步骤

- ◆ C语言中对文件的操作几乎都是通过文件处理函数来实现的，一般以f开头的函数均为文件处理函数，可以分为以下几类：
 - 文件打开与关闭函数（**fopen, fclose**）
 - 文件读写函数（**fgetc/fputc, fread/fwrite, fscanf/printf**等）
 - 文件定位函数（**rewind, fseek**等）
 - 文件检测函数（**feof, ferror**等）
- ◆ 文件操作的4个基本步骤
 - 步骤1：利用“**FILE**”定义文件类型指针。
 - 步骤2：打开文件。
 - 步骤3：对文件进行读或写操作。
 - 步骤4：关闭文件。

8.1.3 文件操作的基本步骤

【例8.1】将26个大写字母'A'—'Z'写入文件保存起来

```
#include <stdio.h>
#include <stdlib.h>
int main( )
{ char ch;
  FILE *fp;           //定义文件指针变量；步骤1
  fp=fopen("letter.dat","w"); //新建并打开一个文件；步骤2
  if(fp==NULL)         //判断文件打开成功与否
  { printf("\n Opening file error");
    exit(0);
  }
  for (ch='A'; ch<='Z'; ch++)
    fputc(ch, fp);     //将变量ch中的字符写入文件；步骤3
  fputc('\n',fp);      //最后写入一个换行符
  fclose(fp);          //关闭文件；步骤4
  return 0;
}
```

8.2 文件的打开与关闭

- 在进行文件读写操作之前要先打开文件，读写操作结束后要关闭文件。打开文件实际上是建立文件的各种有关信息，并使文件指针变量指向该文件，以便进行其它操作。

8.2.1 文件打开函数

函数原型: ***FILE *fopen (const char *path, const char *mode);***

函数参数: **const char *path**——文件名称，用字符串表示
const char *mode——文件打开方式，用字符串表示

函数返回值: **FILE**类型指针。如果成功打开文件，**fopen** 返回文件的地址，否则返**NULL**。如果文件打开失败，继续操作是没有意义的，这时通常会给出一个“文件打开失败”的提示信息，然后退出程序。

8.2.1 文件打开函数

- **fopen**函数的使用格式

文件指针变量=**fopen** (文件名, 文件打开方式);

例 **FILE *fp ;**

fp = fopen (“file1.txt” , “r”);

需要打开的文件名

让文件指针变量
指向被打开的文件

文件打开方式(只读)

说明：若要打开的文件在当前目录下，可以只使用文件名；
若不在当前目录下，则应该给出文件的路径。

例： **fp=fopen(“E:\\test\\file8_1.c”, "r");**

8.2.1 文件打开函数

- 打开文件时，因使用方式不对或其他原因，可能会使打开文件操作失败，这时fopen函数的返回值是NULL，所以在打开文件时通常会判断一下打开操作是否成功

```
FILE *fp;
```

```
if ( ( fp = fopen("file1.txt", "r") ) == NULL )  
{  
    printf("Can not open this file!\n");  
    exit( 0 );  
}
```

说明: **exit** 函数的作用是 结束程序, 并返回操作系统

注意: 使用**exit**函数要包含头文件 **<stdlib.h>**

8.2.1 文件打开函数

◆文件打开方式

(1) “r”与“rb” 只读

“r”打开一个文本文件,“rb”打开一个二进制文件

以此方式打开文件, 该文件必须已经存在, 如不存在将出错。

打开文件后只能从文件中读数据, 当前读写位置位于文件开头

(2) “w”与“wb” 只写

“w”打开一个文本文件,“wb”打开一个二进制文件

以此方式打开文件时, 如有同名文件(即文件已存在), 则将原有文件删除并建立一个新文件; 如没有同名文件(即文件不存在), 则建立一个新文件。

打开文件后只能向文件中写数据, 当前读写位置位于文件开头

8.2.1 文件打开函数

(3) “a”与“ab” 追加

以此方式打开文件时, 如文件已存在, 则将当前读写位置位于文件末尾, 可以追加数据; 如文件不存在, 则建立一个新文件。

(4) “r+”与“rb+” “+”表示即能读又能写

用此方式打开文件时, 文件必须已经存在, 否则将出错

(5) “w+”与“wb+” 以读/写方式打开文件, 建立一个新文件。 若文件已存在, 则将原文件覆盖。

(6) “a+”与“ab+”

以读/写方式打开文件, 如文件已存在, 可在文件末尾追加数据; 若文件不存在, 则建立一个新文件

说明: 以“a+”或“ab+”打开已经存在的文件, 若打开文件后直接写数据, 则新数据添加在文件的末尾; 若打开文件后直接进行读数据, 则是从文件开头读取数据

8.2.1 文件打开函数

使用方式		含 义
文本文件	“r”	以只读方式打开一个文本文件
	“r+”	以读/写方式打开一个文本文件
	“w”	以只写方式建立一个新的文本文件
	“w+”	以读/写方式建立一个新的文本文件
	“a”	向文本文件末尾追加数据
	“a+”	以读/写方式打开一个文本文件
二进制文件	“rb”	以只读方式打开一个二进制文件
	“rb+”	以读/写方式打开一个二进制文件
	“wb”	以只写方式建立一个新的二进制文件
	“wb+”	以读/写方式建立一个新的二进制文件
	“ab”	向二进制文件末尾追加数据
	“ab+”	以读/写方式打开一个二进制文件

8.2.2 文件关闭函数

- 关闭文件是指断开文件指针变量与文件之间的联系，即禁止文件指针变量再对该文件进行操作。

函数原型: *int fclose(FILE *stream);*

函数参数: FILE **stream*——打开文件的地址

函数返回值: 正常关闭时将返回0值，否则返回非0值。

- **fclose**函数的重要作用:

写文件时，系统是在“输出缓冲区”满时，才将数据写入文件。当程序结束时，最后几次“写文件”的数据可能还在缓冲区中，而缓冲区还未满，如果未关闭文件就结束程序，则缓冲区的数据不能写入文件，会丢失数据。使用**fclose**函数，不论缓冲区是否已满，都会将缓冲区的数据写入文件，再关闭文件，从而保证文件数据的完整性。

【例8.2】 已知一个文本文件 “file8_1.c”存储在 “E:\test”目录中，
将该文件的信息输出到显示器上，

```
#include <stdio.h>
#include <stdlib.h>
int main( )
{   FILE *fp;
    char ch;    int i;
    if ((fp=fopen("E:\\test\\file8_1.c", "r"))==NULL)
    {   printf("文件打开失败！ \n");
        exit(0);    }
do
{   ch=fgetc(fp);    //从文件中读取一个字符赋给变量ch
    putchar(ch);    //将变量ch中的字符输出到显示器
} while(ch!=EOF); //当ch!=EOF时执行循环体
fclose(fp);    //关闭文件
return 0;
}
```

8.3 文件的读写

8.3.1 字符读写函数

1. 字符读取函数

函数原型: *int fgetc(FILE *fp);*

函数参数: **FILE *fp** — 文件指针

函数功能: 从**fp**所指向的文件中读取一个字符, 字符由函数返回。返回的字符可以赋值给字符变量, 也可以直接参与表达式运算。 每次读取一个字符后, 文件位置指针自动指向下一个字节。

函数返回值: 输入成功返回输入的字符;
遇到文件结束, 返回**EOF(-1)**。

8.3.1 字符读写函数

2. 字符写入函数

函数原型: *int fputc(int ch, FILE *fp);*

函数参数: **int ch** — 要写入文件的字符
FILE *fp — 文件指针

函数功能: 将**ch** (可以是字符常量、变量或字符表达式等) 写入**fp**所指向的文件。每次写入一个字符, 文件位置指针自动指向下一个字节。

函数返回值: 输出成功, 返回输出的字符**ch**;
输出失败, 返回**EOF**。

【例8.3】 将磁盘上一个文本文件的内容复制到另一个文件中，源文件和目标文件的文件名在程序运行时输入。

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    FILE *fp_s, *fp_d;  char ch, sfile[20], dfile[20];
    printf("Enter the source filename:");
    scanf("%s", sfile);          //输入源文件的文件名
    printf("Enter the destination filename: ");
    scanf("%s", dfile);          //输入目标文件的文件名
    if((fp_s=fopen(sfile,"r"))==NULL)  //以只读方式打开源文件
    { printf("Can't open file: %s", sfile); exit(0); }
    if((fp_d=fopen(dfile,"w"))==NULL)  //以只写方式打开目标文件
    { printf("Can't open file: %s", dfile); exit(0); }
    ch= fgetc(fp_s);      //从源文件读一个字符，并将它赋值给变量ch
    while(ch!=EOF)        //当源文件未结束时进行循环
    { fputc(ch, fp_d);     //将变量ch中存放的字符写入目标文件
      ch= fgetc(fp_s);
    }
    fclose(fp_s);         //关闭源文件
    fclose(fp_d);         //关闭目标文件
    return 0;
}
```


8.3.1 字符读写函数

【例8.4】从键盘输入一行字符，写入到文本文件“string.txt”中。

```
#include <stdio.h>
#include <stdlib.h>
int main( )
{ FILE *fp;    char ch;
  if ( (fp=fopen("string.txt", "w"))==NULL) //只写方式打开文件
  { printf("Can't open file!\n");
    exit(0);  }
  do
  { ch=getchar( );    //从键盘读取一个字符
    fputc(ch, fp);    //将字符写入文件
  } while(ch!='\n');  //输入字符不是回车符时执行循环体
  fclose(fp);
  return 0;
}
```

【练习】

编程统计一个文本文件中小写字母a的个数。

（文本文件可以先用记事本写好）

8.3.2 字符串读写函数

1. 字符串读取函数

函数原型: *char *fgets(char *str, int n, FILE *fp)*

函数参数: **char *str** — 存放字符串的存储单元的首地址
int n — 读取字符的个数（包括结束标志在内）
FILE *fp — 文件指针

函数功能: 从fp所指向的文件读n-1个字符, 并将这些字符存放
到以str为起始地址的存储单元中, 在读入的最后一个
字符后自动添加字符串结束标志'\0'。如果在读入n-1
个字符结束前遇到换行符或EOF, 则读入结束。

函数返回值: 读取数据成功时, 返回指针str;
遇到文件结束或出错, 则返回NULL。

8.3.2 字符串读写函数

【例8.5】编程将一个文本文件中全部信息显示到屏幕上

```
#include <stdio.h>
#include <stdlib.h>
int main( )
{ FILE *fp;    char str[81], filename[20];
  printf("输入文件名:");
  gets(filename);
  if((fp=fopen(filename, "r"))==NULL) //只读方式打开文本文件
  {   printf("can't open file!\n");    exit(0);   }
  while( fgets(str,81,fp)!=NULL ) //读取字符串成功时执行循环
      printf(" %s", str);    //将字符串输出到显示器上
  fclose(fp);
  return 0;
}
```

8.3.2 字符串读写函数

2. 字符串写入函数

函数原型: *int fputs(char *str, FILE *fp)*

函数参数: **char *str** — 存放字符串的存储单元的首地址
FILE *fp — 文件指针

函数功能: 向**fp**所指向的文件写入一个字符串**str**, 其中字符串可以是字符串常量, 也可以是字符数组名, 或字符指针变量, **注意字符串结束标志'\0'是不写入文件的。**

函数返回值: 写入成功时返回**0**; 出错时返回非**0**值。

8.3.2 字符串读写函数

【例8.6】 用fputs函数在文本文件“string.txt”末尾添加若干行字符

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main( )
{ FILE *fp;    char str[81];
  if((fp=fopen("string.txt", "a"))==NULL) //追加方式打开文件
  { printf("Can't open file!\n");  exit(0);  }
  while( strlen(gets(str))>0 )
  { fputs(str, fp);    //将字符串写入文件
    fputs("\n", fp);   //补写一个换行符
  }
  fclose(fp);
  return 0;
}
```

8.3.3 数据块读写函数

- 字符和字符串读写函数适用于文本文件，对于二进制文件，通常使用数据块读写函数 **fread/fwrite**。

函数原型：

```
int fread ( void *buffer, int size, int count, FILE *fp);  
int fwrite ( void *buffer, int size, int count, FILE *fp);
```

函数参数：

void *buffer — 对 **fread** 来说，它是用于存放读入数据的首地址；
对 **fwrite** 来说，它是要输出数据的首地址。

int size — 表示一个数据块的字节数，即一个数据块的大小。

int count — 要读写的数据块的个数。

FILE *fp — 文件指针。

8.3.3 数据块读写函数

*int fread (void *buffer, int size, int count, FILE *fp);*

*int fwrite (void *buffer, int size, int count, FILE *fp);*

fread函数的功能：从文件fp当前位置指针处中读取count个长度为size字节的数据块，存放到内存中buffer所指向的存储单元中，同时读写位置指针后移count*size个字节。

fwrite函数的功能：将内存中buffer所指向的存储单元中的数据写入文件fp中，每次写入长度为size字节的count个数据块，同时读写位置指针后移count*size个字节。

函数返回值：如果读取/写入操作成功，函数返回值等于实际读取/写入的数据块的个数(即返回值等于count值)；若出现错误, 或已到文件尾, 则返回值小于count值。

【例8.7】 从键盘输入10个整数, 将它们保存到文件 “int_file.dat”中;
然后从文件中读取这10个整数输出到显示器上。

```
#include <stdio.h>
#include <stdlib.h>
int main ( )
{ FILE *fp ;
  int n, i ;
  if ( ( fp=fopen("int_file.dat", "wb")) == NULL )
  { printf("Can't open file!\n");    exit(0);    }
  for ( i=0; i<10; i++ )
  { scanf("%d", &n);    //从键盘输入一个整数, 存放到变量n中
    fwrite(&n, sizeof(int), 1, fp); } //将变量n中的数据写入文件
  fclose (fp);          //关闭文件
  if ( ( fp=fopen("int_file.dat", "rb")) == NULL ) //重新打开文件
  { printf("Can't open file!\n");    exit(0);    }
  for ( i=0; i<10; i++ )
  { fread (&n, sizeof(int), 1, fp); //从文件读取一个整数存放到变量n中
    printf("%4d", n ) ; } //将变量n中的数据输出到显示器
  fclose (fp) ;
  return 0;
}
```

```
#include<stdio.h>
#include<stdlib.h>
int main ( )
```

```
{ FILE *fp ;
```

```
int a[10], b[10]={0}, i ;
```

```
if ( ( fp=fopen("int_file.dat", "wb")) == NULL )
```

```
{ printf("Can't open file!\n"); exit(0); }
```

```
for ( i=0; i<10; i++ ) scanf(" %d", &a[i] );
```

```
fwrite( a, sizeof(int), 10, fp ); //将数组a中的10个数据一次写入文件
```

```
fclose (fp);
```

```
if ( ( fp=fopen("int_file.dat", "rb")) == NULL ) //重新打开文件
```

```
{ printf("Can't open file!\n"); exit(0); }
```

```
fread ( b, sizeof(int), 10, fp ); //从文件中一次读10个数据存放在数组b中
```

```
for ( i=0; i<10; i++ ) printf(" %4d", b[i] );
```

```
printf("\n");
```

```
fclose (fp) ;
```

```
return 0;
```

```
}
```

【例8.7】方法2：使用一个整型数组，先通过循环从键盘输入10个整数，再将这10个整数一次写入文件；从文件读取数据时，一次读取10个整数存到数组中，再通过循环将它们输出到显示器上。

8.3.4 格式化读写函数

函数格式:

*fprintf (FILE *fp, 格式控制字符串, 输出列表);*

*fscanf (FILE *fp, 格式控制字符串, 变量地址列表);*

函数功能:

fprintf函数-- 将输出列表中的数据按指定格式写到**fp**所指向的文件中, 返回值为实际写入文件的字符数;
若输出失败, 返回**EOF**。

fscanf函数 -- 按指定格式从**fp**所指向的文件中读取数据, 送到对应的变量中。返回值为所读取的数据项个数;
若读取操作失败或文件结束, 返回**EOF**。

【例】将1-5的平方值按一定的格式存入文件，再从文件中读出输出到显示器上

```
#include <stdio.h>
#include <stdlib.h>
int main( )
{   FILE *fp;
    int i, a[5], b[5]={0}, c[5]={0};
    if ((fp=fopen("list.dat","w"))==NULL)
        {   printf("Can't open file!\n");    exit(0);   }
    for(i=0;i<5;i++)
        {   a[i]=(i+1)*(i+1);
            fprintf(fp,"%d,%d\n", i+1, a[i]);
        }
    fclose(fp);
    if ((fp=fopen("list.dat","r"))==NULL)
        {   printf("Can't open file!\n");    exit(0);   }
    for(i=0;i<5;i++)
        {   fscanf(fp,"%d,%d", &b[i], &c[i]);
            printf("%d  %d\n",b[i],c[i]);
        }
    fclose(fp);
    return 0;
}
```

【例8.9】下面程序将生成“list.dat”文件，在该文件中以“单价,数量”格式存放了某商场的商品数据(商品的单价和数量分别存放在两个一维数组中)，程序将计算出所有商品的总金额，并将它存放在“list.dat”文件的最后一行。

```
#include <stdio.h>
#include <stdlib.h>
int main( )
{   FILE *fp;
    float total=0, price, p[3]={12.3, 45.6, 78.9};
    int i, num, n[3]={10, 20, 30};
    if ((fp=fopen("list.dat","w"))==NULL)
    {   printf("Can't open file!\n");   exit(0);   }
    for(i=0; i<3; i++)
    {   fprintf(fp, "%5.2f, %d\n", p[i], n[i]); //按格式写入商品数据
        total =total+p[i]*n[i];                //计算所有商品的总金额
    }
    fprintf(fp,“总金额=%f\n”, total);        //按格式写入总金额
    fclose(fp);
    ...
}
```

【例8.9】参考程序

```
int main( )
```

```
{  ...  
    if ((fp=fopen("list.dat","r"))==NULL)    //重新打开文件  
    {  printf("Can't open file!\n");    exit(0);  }  
    for(i=0; i<3; i++)  
    {  fscanf(fp,"%f, %d", &price, &num);    //按格式从文件中读取数据  
        printf("%f , %3d\n", price, num);    //输出数据到显示器  
    }  
    fscanf(fp,"总金额=%f", &total);    //按格式从文件中读取数据  
    printf("总金额=%8.2f\n", total);    //输出数据到显示器  
    fclose(fp);  
    return 0;  
}
```

说明:

fprintf/fscanf函数与printf/scanf函数的作用基本相同，区别在于：fprintf/fscanf读写的对象是磁盘文件，而printf/scanf读写的对象是默认终端(显示器和键盘)。

8.4 文件的定位

- 目前我们讲过的文件读写是[顺序读写](#)，打开文件后，从文件的开头依次读写数据，实际上，文件中有一个文件位置指针，对当前指向的数据项进行读/写操作后，文件位置指针会自动指向下一个数据项。
- 有时我们需要修改文件中的某个数据，希望能将文件位置指针直接指向要修改的数据，即移动文件位置指针到读写的位置，再进行读写，这就是文件的[随机读写](#)。
- 实现文件的随机读写，就需要对文件位置指针进行设置，即文件的定位，可以通过[文件定位函数](#)来实现。

8.4.1 复位函数

函数原型: ***void rewind(FILE *fp);***

函数参数: **FILE *fp** — 文件指针

函数功能: 将文件位置指针重新定位到文件开始的地方。

说明:

在前面的例题中, 有时需要先对文件进行“写”操作, 再对文件进行“读”操作, 原来我们采用的方法是: 先以“只写”方式打开文件, 写完数据后关闭文件, 再以“只读”方式打开文件, 读取后再关闭文件, 这种方法显然比较麻烦。

改进方法: 先以“读/写”方式打开文件, 进行读/写操作后, 使用 **rewind** 函数, 将文件位置指针重新指向文件的开头, 然后再对文件进行其它读/写操作。

【例8.7】程序代码

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main ( )
```

```
{ FILE *fp ;
```

```
int n, i ;
```

```
if ( ( fp=fopen("int_file.dat", "wb")) == NULL )
```

```
{ printf("Can't open file!\n"); exit(0); }
```

```
for ( i=0; i<10; i++ )
```

```
{ scanf("%d", &n);
```

```
fwrite(&n, sizeof(int), 1, fp); }
```

```
fclose(fp); //关闭文件
```

```
if ( ( fp=fopen("int_file.dat", "rb")) == NULL ) //重新打开文件
```

```
{ printf("Can't open file!\n"); exit(0); }
```

```
for ( i=0; i<10; i++ )
```

```
{ fread (&n, sizeof(int), 1, fp); //从文件读取一个整数存放到变量n中
```

```
printf("%4d", n ); } //将变量n中的数据输出到显示器
```

```
fclose(fp);
```

```
return 0;
```

```
}
```

改为“wb+”

改为rewind(fp);

用rewind函数修改后的例8.7的程序代码:

```
#include <stdio.h>
#include <stdlib.h>
int main ( )
{ FILE *fp ;
  int n, i ;
  if ( ( fp=fopen("int_file.dat", "wb+")) == NULL )
  { printf("Can't open file!\n");      exit(0);      }
  for ( i=0; i<10; i++ )
  { scanf("%d", &n);
    fwrite(&n, sizeof(int), 1, fp); }
    rewind(fp);                //调用复位函数
  for ( i=0; i<10; i++ )
  { fread (&n, sizeof(int), 1, fp);
    printf("%4d", n ) ; }
    fclose (fp) ;
    return 0;
}
```



8.4.2 随机移动函数

函数原型: *int fseek(FILE *fp, long offset, int base);*

函数参数:

FILE *fp — 文件指针

long offse — 表示以起始点为基准, 向前或向后移动的字节数。
向前移动是指向“文件开头”移动, 用负数表示;
向后移动是指向“文件末尾”移动, 用正数表示。

int base — “起始点”, 表示从何处开始计算位移量。

起始点	表示符号	数字表示
文件开头	SEEK_SET	0
当前位置	SEEK_CUR	1
文件末尾	SEEK_END	2

8.4.2 随机移动函数

函数原型: *int fseek(FILE *fp, long offset, int base);*

函数功能: 将文件位置指针移到以base为起始点,
以offset为位移量的位置。

函数返回值: 移动成功返回0, 失败返回非0值。

例如:

fseek(fp, 20L, 0); 文件位置指针从文件开始向后移动20个字节

fseek(fp, 10L, 1); 文件位置指针从当前位置向后移动10个字节

fseek(fp, -30L, 2); 文件位置指针从文件末尾向前移动30个字节

注意: 因第2个参数为长整型, 所以在整数后加字母 ‘L’

1. 从键盘输入10个学生的信息（学生信息包括姓名、学号和成绩），将信息保存在二进制文件“student1.dat”中，然后再从文件中读取学生信息，并这些信息输出到显示器上。
2. 从键盘输入10个学生的信息（学生信息包括姓名、学号和成绩），将信息保存在二进制文件“student2.dat”中，然后再从文件中读取序号为奇数的学生信息，并这些信息输出到显示器上。
3. 测指定二进制文件的长度
4. 统计文本文件中字符的个数

1. 从键盘输入10个学生的信息（学生信息包括姓名、学号和成绩），将信息保存在二进制文件“student1.dat”中，然后再从文件中读取学生信息，并这些信息输出到显示器上。
2. 从键盘输入10个学生的信息（学生信息包括姓名、学号和成绩），将信息保存在二进制文件“student2.dat”中，然后再从文件中读取序号为奇数的学生信息，并这些信息输出到显示器上。
3. 测指定二进制文件的长度
4. 统计文本文件中字符的个数

【例8.11】 从键盘输入6个学生的信息（学生信息包括姓名、学号和成绩），将信息保存在二进制文件“student.dat”中，然后再从文件中读取序号为奇数的学生信息，并这些信息输出到显示器上。

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define SIZE 6      //定义符号常量SIZE表示学生人数
```

```
typedef struct student
```

```
{ char name[12];
```

```
    int num;
```

```
    float score;
```

```
}STUD;              //定义结构体类型STUD
```

```
long len=sizeof(STUD); //全局变量len设初值
```

```
void save(STUD a[ ]); //函数功能是将结构体数组中的数据写入文件
```


【例8.11】 参考程序

```
void save(STUD a[ ])    //定义函数，将结构体数组中数据写入文件
{ FILE *fp;
  if((fp=fopen("student.dat", "wb"))==NULL)
  { printf("Can't open file!\n");
    exit(0);
  }
  fwrite(a, len, SIZE, fp); //将数组a中的SIZE个数据一次写入文件
  fclose(fp);
}
```

【例8.11】 参考程序

```
int main( )
```

```
{ FILE *fp;      STUD st[SIZE], x;          int i;
  printf("请按姓名,学号,成绩的顺序输入%d个学生的信息:\n", SIZE);
  for (i=0; i<SIZE; i++)
  { printf("学生%d: ", i+1);
    scanf("%s%d%f", st[i].name, &st[i].num, &st[i].score);    }
  save(st);    //调用函数将数据保存到文件中, 用结构体数组名作参数
  if ((fp=fopen("student.dat", "rb"))==NULL)
  { printf("Can't open file!\n");    exit(0);    }
  printf("输出学生信息: \n");
  for(i=0; i<SIZE; i=i+2)
  { fseek(fp, i*len, 0); //文件位置指针从文件开头向后移动i*len个字节
    fread(&x, len, 1, fp);
    printf("学生%d: %s, %d, %.2f\n", i+1, x.name, x.num, x.score);
  }
  fclose(fp);
  return 0;
}
```

8.4.3 取当前位置函数

函数原型: *long ftell(FILE *fp);*

函数参数: **FILE *fp** — 文件指针

函数功能: 返回文件位置指针相对于文件开头的位移量。

函数返回值: 返回文件位置指针距离文件开始处的字节数,
若出错则返回-1L。

【例8.12】测指定二进制文件的长度。

```
#include <stdio.h>
#include <stdlib.h>
int main( )
{   FILE *fp;
    char filename[50];
    puts("输入一个文件名:");
    gets(filename);
    if ((fp=fopen(filename, "rb"))==NULL)
    {   printf("Can't open file!\n");
        exit(0);
    }
    fseek(fp, 0, 2);           //移动文件位置指针到文件末尾
    printf("len=%ld\n", ftell(fp)); //输出文件长度（字节数）
    return 0;
}
```

8.5 文件检测函数

1. **feof**函数：用于检测文件位置指针是否已到达文件末尾。

在对文件进行读操作时，常需要测试文件是否已经到了文件末尾，文本文件一般用EOF来进行检测，但EOF不适用于二进制文件，而文件结束检测函数**feof()**既适用于文本文件，也适用于二进制文件。

函数原型：***int feof(FILE *fp)***

函数参数：**FILE *fp** — 文件指针

函数功能：检测**fp**所指向文件中的文件位置指针是否指向文件尾

函数返回值：若文件结束（即文件位置指针指向文件尾），
则返回非0值；否则返回0。

【例8.13】统计文本文件中字符的个数

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main( )
```

```
{ FILE *fp;    int n=0;
```

```
  char ch, filename[20];
```

```
  printf("Enter the file name:");
```

```
  gets(filename);
```

```
  if((fp=fopen(filename, "r"))==NULL) //只读打开文件
```

```
  { printf("can't open file!\n");  exit(0); }
```

```
  while( !feof(fp) )
```

```
  { ch=fgetc(fp);
```

```
    putchar(ch);
```

```
    n++;
```

```
  }
```

```
  printf("\n字符个数=%d\n", n);
```

```
  fclose(fp);
```

```
  return 0;
```

```
}
```

若不想统计EOF，修改方法1:

```
while( (ch=fgetc(fp))!=EOF )
```

```
{  putchar(ch);
```

```
   n++;
```

```
}
```

将文本文件的结束
标志EOF也统计了

修改方法2:

```
ch=fgetc(fp);
```

```
while( !feof(fp) )
```

```
{  putchar(ch);
```

```
   n++;
```

```
   ch=fgetc(fp);
```

```
}
```

【例8.14】输出二进制文件中的数据(使用例8.11中产生的文件student.dat)

```
#include <stdio.h>
#include <stdlib.h>
#define SIZE 5
typedef struct
{ char name[10];
  int num;
  float score;
}STUD;
long len=sizeof(STUD);
int main( )
{ FILE *fp;
  STUD x; int i=0;
  if ((fp=fopen("student.dat", "rb"))==NULL)
  { printf("Can't open file!\n"); exit(0); }
  printf("输出学生信息: \n");
  while(!feof(fp))
  { fread(&x, len, 1, fp);
    i++;
    printf("学生%d: %s, %d, %.2f\n", i, x.name, x.num, x.score);
  }
  fclose(fp);
  return 0;
}
```

```
输出学生信息:
学生1: mary, 11, 78.00
学生2: john, 22, 65.00
学生3: mike, 33, 90.00
学生4: bob, 44, 88.00
学生5: alex, 55, 99.00
学生6: alex, 55, 99.00
Press any key to continue_
```

出现这种问题的原因：**feof**实际观察的是上次读操作的内容，也就是上次**fread**函数的返回值，若**fread**返回值为1,表明读取的不是空，则**feof**就返回0，表示文件没结束；如果上次读操作失败了，则**feof**返回1，表示文件结束。

修改方法1:

```
fread(&x, len, 1, fp);  
while(!feof(fp))  
{ i++;  
    printf("学生%d: %s, %d, %.2f\n", i, x.name, x.num, x.score);  
    fread(&x, len, 1, fp);  
}
```

修改方法2:

```
while( fread(&x, len, 1, fp)==1 )  
{ i++;  
    printf("学生%d: %s, %d, %.2f\n", i, x.name, x.num, x.score);  
}
```


8.5 文件检测函数

2. **ferror**函数

在调用各种文件读写函数时,可能因某些原因导致失败,如果出现错误,除了函数返回值有所反映外,还可用**ferror**函数进行检查

函数原型: **int ferror(FILE *fp)**

函数参数: **FILE *fp** — 文件指针

函数功能: 检查**fp**所指向的文件在调用各种读写函数时是否出错。

函数返回值: 若当前读写操作没有出现错误, 则返回**0**;

出错则返回非**0**值。

说明:

① 对同一文件每调用一次读写函数,均产生一个新的**ferror**函数值,因此,应当在调用一个读写函数后立即检查**ferror**函数的值,否则信息会丢失。

② 在执行**fopen**函数时, **ferror**函数的初始值自动置为**0**。

8.5 文件检测函数

3. `clearerr`函数

当错误处理完毕后，应清除相关的错误标志，以免进行重复的错误处理，这时应使用`clearerr`函数。

函数原型：**`void clearerr(FILE *fp)`**

函数参数：**`FILE *fp`** — 文件指针

函数功能：使文件错误标志和文件结束标志置为0。假设在调用一个读写函数时出现错误，`ferror`函数值为非零值。在调用`clearerr(fp);`后，`ferror`函数值变成0。

说明：

一旦文件读写操作出现错误，系统内部的一个错误标志就被设为非0值，调用`ferror()`函数可得到该错误标志的值。错误标志会一直保留，直到清除调用`clearerr`函数，或下一次调用读写函数，才能改变该标志的值。

【例8.15】： clearerr应用举例

```
#include <stdio.h>
#include <stdlib.h>
int main( )
{ FILE *fp;    char ch;
  if((fp = fopen("newfile.txt", "w"))==NULL)
  {   printf("Can not open file.\n");  exit(0);  }
  ch=fgetc(fp);
  if (ferror(fp))    //出错返回非0值
  {   printf("Error reading !\n");
      clearerr(fp); //调用后ferror(fp)的值为0
  }
  if( !ferror(fp) )
  printf("Error indicator cleared!\n");
  fclose(fp);
  return 0;
}
```

本章小结

- ❖ C语言把文件看作是一个字节的序列。
- ❖ 根据数据的组织形式文件分为两类：文本文件和二进制文件。
- ❖ C中通过一个文件类型的指针使用文件。
- ❖ 使用文件的第一步用fopen()函数打开指定文件，打开时必须指明文件的打开方式，使用文件的最后一步是关闭文件，通过fclose()函数实现。
- ❖ 文件读写是通过各类读写函数实现的。
 - (1) 读写一个字符用fgetc()/fputc();
 - (2) 读写一个字符串用fgets()/fputs();
 - (3) 读写一个数据块用fread()/fwrite();
 - (4) 以特定格式读写数据时，用fscanf()/fprintf()函数。
- ❖ 文件随机读写需要借助文件定位函数rewind()、fseek()、ftell()
- ❖ 常用的文件检测函数包括：feof()、ferror()、clearerr()



培养创新思维 提升编程能力